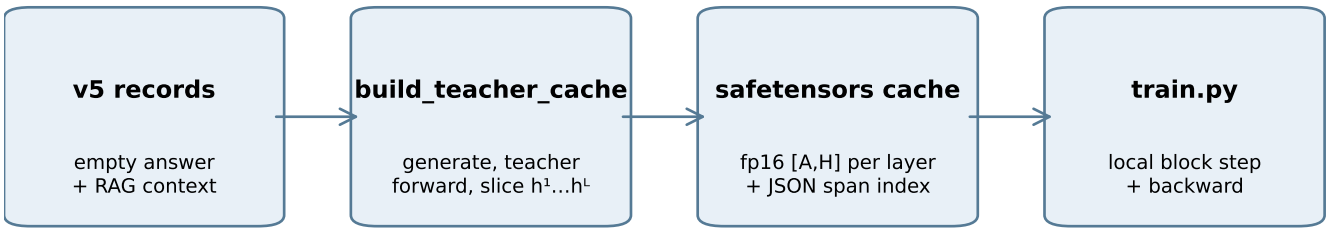
The v5 training

A source-backed map of cache construction and layerwise training

What this note establishes

The v5 pipeline separates teacher computation (once, into a disk cache) from student training (many local backward steps). The cache is cheap in GPU terms because it remains on disk/host until a batch needs it. The current 1.7B OOM is instead consistent with transient full-vocabulary divergences.



Scope

This is a code-reading document, not a claim that the present loss implementation is optimal. It reports what the active v5 configuration and source do today, then identifies which tensor shapes make the current full-vocabulary objective memory-sensitive.

```
scripts/train.py:20–36

cfg = load_config(args.config, args.experiment)
...
run_dir = train_layerwise(cfg)
```

Current active setting

The queued 1.7B jacobian-lens-KL runs use summed scheduling, connected-window width 1, bucketed batches of four, 16-example gradient accumulation, a frozen teacher copy, and offloaded Adam moments. The 0.6B base uses batches of eight and eight-example accumulation.

1. Building the frozen-teacher cache

The teacher is run once per record; its aligned hidden states become a model-specific artifact.

Open-answer v5 records

The dataset has empty answer fields. For each record, the teacher first greedily generates an answer with the RAG context present. Those generated token ids are cache content, rather than reference-text training targets. The answer is then included in the teacher-forced forward whose hidden states are saved.

scripts/build_teacher_cache.py:164–174

```
if v5:
    answer_ids, hard_cut = generate_answer(model, masker, ex, stop_id, budget)
    pair = masker.build(ex, answer_ids=answer_ids)
    extra = {"answer_ids": answer_ids, "hard_cut": hard_cut}
```

Teacher forward and span restriction

The builder requests all transformer hidden states, but writes only the aligned span. For an open-answer record that span is the shared middle plus the teacher-generated answer, not the privileged RAG evidence itself.

scripts/build_teacher_cache.py:186–203

```
out = model(t_ids, output_hidden_states=True, use_cache=False)
span = pair.t_aligned
hidden = {
    L: out.hidden_states[L][0, span.start:span.stop]
    for L in range(1, n_layers + 1)
}
writer.add(ex.example_id, hidden, span={..., "A": pair.aligned_len, ...})
```

Storage contract

Each layer is detached, converted to the configured storage dtype (the active caches are float16), made contiguous, moved to CPU, and written in safetensors shards. The index carries alignment and generated-answer metadata. Training subsequently opens tensors lazily by example and layer.

src/selfupdate/teacher/cache.py:111–127, 158–167

```
stored = h.detach().to(self.hidden_dtype).contiguous().cpu()
self._buffer[f"{example_id}/h{L:02d}"] = stored
```

2. What the cache costs

Exact sizes from the active cache indexes and model configurations; fp16 hidden states only.

model	layers × hidde	fp16 cache / aligned to	examples	aligned tokens	cache payload
0.6B	28 × 1,024	56 KiB	2071	305,189	16.30 GiB
1.7B	28 × 2,048	112 KiB	2071	316,490	33.80 GiB

Formula

For L layers, hidden width H, aligned length A, and fp16 storage, the hidden-state payload is L × H × A × 2 bytes. This is disk/host storage; it is not a simultaneous GPU allocation during normal cached training.

```
src/selfupdate/teacher/cache.py:8–18

Per example, restricted to the aligned span (length A):
- ``h{L}``   [A, H] float16
```

Active 1.7B cache

The active 1.7B remove-compaction cache contains 316,490 aligned tokens. At 112 KiB per aligned token, its raw hidden payload is 33.80 GiB before safetensors/index overhead. The 0.6B cache is 56 KiB/token and 16.30 GiB raw.

3. Current span distribution

Long spans drive transient vocabulary-loss memory; the values below are read from each cache index.

model	span	min	p50	p90	p99	max
0.6B	aligned (hidden los	50	123	243	436	732
0.6B	answer (readout)	4	59	135	328	673
1.7B	aligned (hidden los	49	130	262	479	962
1.7B	answer (readout)	7	66	143	373	841

Why there are two lengths

The hidden loss is evaluated on the full aligned span A: shared middle plus answer. The readout teacher-KL starts at the answer and therefore has A − mid_len rows. Both are variable-length; bucketed batching reduces padding but deliberately retains a real batched forward/backward.

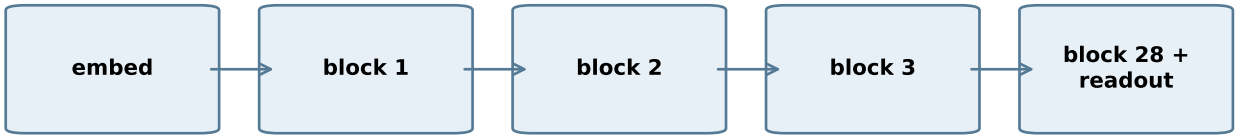
```
src/selfupdate/data/dataset.py:215–250

Amax = max(it.A for it in items)
ans_lens = [it.s0 + it.A - it.ans0 for it in items]
Rmax = max(ans_lens) if ans_lens else 0
...
readout_index[i, :rlen] = torch.arange(it.ans0 - 1, it.s0 + it.A - 1)
```

Important interpretation

A long example contributes many tokenwise terms to the same example-average KL. It does not create one optimizer update per token. It does, however, cause a larger vocabulary-shaped intermediate when all positions are decoded together.

4. What train.py does for the active slide-1 runs

The current schedule is local in depth; it does not retain graphs for all 28 layers until epoch end.



detach before each local step → backward → retain only detached output

Slide width one

With conn_window = 1, the summed loop calls one local block step per layer. The local helper runs that block, forms the layer loss, calls backward, and returns detached tensors. The final one-block window combines final hidden loss with the sanctioned teacher-KL readout and then calls backward once.

src/selfupdate/train/layerwise.py:454–466

```
loss_vals, h = local_block_step_batch(
    stack, L, h.detach(), pos_emb, target,
    batch, loss_fn, previous_target=targets.get(L - 1),
)
layer_losses.append(loss_vals)
L += 1
```

src/selfupdate/train/steps.py:168–175

```
h_out = stack.run_block(L, h_in, pos_emb)
losses = _layer_loss_per_example(...)
total = losses.sum()
...
total.backward()
return losses.detach(), h_out.detach()
```

5. The vocabulary-shaped local-loss peak

This is an implementation property of lens-KL objectives, not the disk-cache footprint.

The operation

jacobian_lens_kl transports a hidden state through a frozen Jacobian and then uses the lens-KL fallback. The lens decodes student and teacher hidden rows through the frozen LM head, converts both logit tensors to fp32 for log-softmax, and evaluates KL over all 151,936 vocabulary entries.

```
src/selfupdate/train/losses.py:390–410

s_logits = self.lm_head(student_h)
with torch.no_grad():
    t_logits = self.lm_head(teacher_h)
return F.kl_div(
    F.log_softmax(s_logits.float(), dim=-1),
    F.log_softmax(t_logits.float(), dim=-1),
    log_target=True, reduction="batchmean",
)
```

Per-example loop

The padded-batch helper slices each example to its real prefix, but appends all per-example loss graphs before summing and calling backward. Therefore the full-vocabulary intermediates for up to the micro-batch size coexist during one local layer. They are released after that local backward, not retained across all depth layers.

```
src/selfupdate/train/steps.py:75–89, 152–175

for i, k in enumerate(lens):
    losses.append(loss_fn(student_h[i, :k], teacher_h[i, :k], ...))
return torch.stack(losses)
...
total = losses.sum()
total.backward()
```

6. Concrete 1.7B peak arithmetic

Lower-bound tensor sizes; kernels and autograd can require additional workspace/saved tensors.

aligned rows A	one bf16 logit table	one fp32 vocab table	four fp32 tables
130	38 MiB	75 MiB	301 MiB
479	139 MiB	278 MiB	1110 MiB
962	279 MiB	558 MiB	2230 MiB

How to read the table

One fp32 vocabulary table is A × 151,936 × 4 bytes. At the maximum 962-row aligned span it is 557 MiB. The actual KL path has more than one such conceptual table: student/teacher logits, float conversions, log-softmax outputs, KL work, autograd state, block activations, model weights, gradients, and the frozen teacher copy. The table is therefore not a complete VRAM estimate; it explains why the shape is hazardous.

Observed failure versus diagnosis

The scheduler log reported an OOM while allocating 252 MiB in `log_softmax`/`kl_div`, after epoch-zero evaluation and before the first epoch completed. 252 MiB corresponds to about 435 rows of one fp32 [rows, vocab] table. This rules out a multi-epoch accumulation as the immediate trigger; it does not by itself prove that every transient is optimally released.

src/selfupdate/train/steps.py:137–149

```
for i, k in enumerate(lens):
    losses.append(F.kl_div(
        F.log_softmax(student_logits[i, :k].float(), dim=-1),
        F.log_softmax(teacher_logits[i, :k].float(), dim=-1),
        log_target=True, reduction="batchmean",
    ))
```

7. Active v5 configurations and consequences

Configuration values were read from the current base YAML files.

model	micro-batch	grad accum	examples / optimizer	Adam offload	window
0.6B	8	8	64	no	1
1.7B	4	16	64	yes	1

Gradient scale

A tokenwise KL is reduced by `batchmean` inside each example, and example losses are summed before backward. Thus span length changes the number of token terms and memory/compute cost, but not the intended per-example loss scale. The batch path combines several examples in one backward; it does not make an optimizer update per token.

`configs/experiments/v5rs/base_1p7b.yaml:14–32`

```
micro_batch: 4
grad_accum: 16
batching: bucketed
hidden_loss: huber        # arm overlay selects jacobian_lens_kl
conn_window: 1
readout_window_blocks: 1
readout_source: teacher_kl
frozen_teacher_copy: true
offload_adam: true
```

What the cache does not decide

The cache fixes teacher targets and saves teacher-forward compute. It does not force whole-span vectorization, full-vocabulary KL, a particular position chunk size, or a particular gradient estimator. Those are training-loss implementation choices.

8. Findings and safe next measurements

A concise separation of confirmed facts, inference, and changes that would need numerical re-checking.

Confirmed from code and current artifacts

(1) v5 caches hold fp16 teacher hidden states per aligned token and layer on disk. (2) the active slide-1 trainer backpropagates locally per layer; it does not retain all 28 layer graphs. (3) jacobian_lens_kl and teacher-KL decode full vocabulary distributions in fp32. (4) the per-example helpers construct all rows in a micro-batch before the local backward. (5) the failed 1.7B attempts did not complete epoch one.

Strong implementation inference

The dominant OOM pressure is a local full-vocabulary peak for long spans, on top of resident student, frozen teacher, gradients, and optimizer-paging state. The 252 MiB failed request matches a single roughly 435-row fp32 vocabulary table. Allocator fragmentation was small in the error report, so it is not the primary explanation.

Do not silently change the science

Position chunking with correctly weighted partial losses aims to preserve the full-span objective, but it changes operation order and should be measured with `scripts/train_certify.py`. Position sampling or top-k/approximate KL changes the estimator/objective and needs an explicit experimental arm, not a memory-only patch.

```
AGENTS.md — Training Runtime & Certification

python scripts/train_certify.py --all --out-dir /tmp/$USER/certify_head
# apply intended numerics-preserving change
python scripts/train_certify.py --all --reference-dir /tmp/$USER/certify_head
```

Suggested instrumentation before a code change

Record peak allocated/reserved memory at the start and end of each local block step, with aligned/readout lengths and layer number. This distinguishes a long-span peak from a true retained-allocation path without guessing from a single OOM trace.

Appendix A. Recovered pre-v5 comparison

Temporary detached worktree: /tmp/selfupdate_lw_pre_v5 at 3fb5305 (2026-07-12 00:17:52 +0200).

A necessary correction

The recovered snapshot is not literally cache-free: it already contains a frozen hidden-state TeacherCacheWriter and writes h01…hL safetensors. The material v5 change is that question-only records have no answer in the dataset, so the teacher must generate an answer before the cache forward; answer ids and hard-cut status are then stored in the cache index.

```
/tmp/selfupdate_lw_pre_v5/scripts/build_teacher_cache.py:65–84

for ex in tqdm(examples, desc="teacher forward"):
    pair = masker.build(ex)
    t_ids = torch.tensor([pair.teacher_ids], device=model.device)
    with torch.no_grad():
        out = model(t_ids, output_hidden_states=True, use_cache=False)
    hidden = {L: out.hidden_states[L][0, span.start:span.stop]
              for L in range(1, n_layers + 1)}
    writer.add(ex.example_id, hidden, span={..., "A": pair.aligned_len, ...})
```

Current v5 difference

The current builder detects the all-empty-answer dataset, greedily generates an answer under the RAG-bearing teacher prompt, rebuilds the pair with those ids, and persists them as `extra`. This is what makes a cache meaningful for the v5 question-only dataset: it supplies the teacher-generated answer sequence as well as teacher hidden targets.

```
scripts/build_teacher_cache.py:164–203

if v5:
    answer_ids, hard_cut = generate_answer(model, masker, ex, stop_id, budget)
    pair = masker.build(ex, answer_ids=answer_ids)
    extra = {"answer_ids": answer_ids, "hard_cut": hard_cut}
...
writer.add(ex.example_id, hidden, span={..., "A": pair.aligned_len, ...}, extra=extra)
```

Why this distinction matters

Comparisons should not describe v5 as simply adding a hidden cache. The older implementation already amortized teacher hidden-state forwards. V5 changes target provenance and sequence construction: the teacher's generated, RAG-conditioned answer is a cache artifact, while the student receives a compacted version of that sequence.

Appendix B. Before/after: what actually changed

Source comparison between recovered 3fb5305 and the current tree.

aspect	recovered pre-generation snapshot	current v5
Teacher hidden-state cache	present: hL safetensors	present: same core mechanism
Answer source	record supplies answer to masker	teacher greedy generation supplies answer ids
Answer metadata in cache index	absent	answer_ids + hard_cut stored as extra
Question-only v5 records	not handled by cache builder	explicitly detected and supported
Memory implication	cache avoids teacher hidden forwarc	same, but generated spans can be longer/variable

What did not change

Both versions slice all model layers at the aligned span and write detached fp16 tensors to CPU safetensors. Therefore the basic L × H × A cache-storage arithmetic is not a novel v5 cost. The change to v5 primarily makes A depend on the teacher's generated answer, which creates the variable-span distribution shown earlier in this document.

```
/tmp/selfupdate_lw_pre_v5/src/selfupdate/teacher/cache.py:99–128

for L, h in hidden.items():
    stored = h.detach().to(self.hidden_dtype).contiguous().cpu()
    self._buffer[f"{example_id}/h{L:02d}"] = stored
...
save_file(self._buffer, str(self.root / f"shard-{self._shard_no:05d}.safetensors"))
```

What this cannot establish

The recovered snapshot is a source baseline, not a matched numerical baseline for the active v5 campaign. It should be used to understand provenance and control flow, not to infer a fair performance or memory delta without re-running matched configurations.

Appendix C. Criticism of the current v5 method

The cache is scientifically useful, but it is not free—and the costs should be stated plainly.

1. A new pre-generation stage

Before v5, the cache builder teacher-forced the answer already present in a record. In v5 question-only data, it must first autoregressively generate an answer for every record, then run a second teacher-forced forward to obtain all hidden states. In the current queue this added roughly an hour-scale GPU preprocessing phase per model/compaction cache, before a student can begin training. It is an operational cost and a new failure surface: hard cuts, poor RAG attention, and cache identity all need validation.

`scripts/build_teacher_cache.py:164–188`

```
if v5:
    budget = _generation_budget(...)
    answer_ids, hard_cut = generate_answer(model, masker, ex, stop_id, budget)
    pair = masker.build(ex, answer_ids=answer_ids)
...
out = model(t_ids, output_hidden_states=True, use_cache=False)
```

2. It trades teacher compute for large stored targets

Caching avoids rerunning the teacher's full hidden-state trajectory during student training. But it stores L × H × A fp16 values and encourages full-span objectives. For 1.7B, each aligned token costs 112 KiB on disk; the active cache is several GiB. The longest generated spans also feed directly into the current full-vocabulary local losses, increasing transient VRAM risk.

`src/selfupdate/teacher/cache.py:111–119; src/selfupdate/train/losses.py:390–410`

```
stored = h.detach().to(self.hidden_dtype).contiguous().cpu()
...
s_logits = self.lm_head(student_h)
t_logits = self.lm_head(teacher_h)
F.log_softmax(..., dim=-1)  # full vocabulary
```

3. Cache generation fixes one teacher trajectory

The student is trained on the teacher-generated answer that was fixed at cache-build time. This is stable and replayable, but it cannot ask what the teacher would do after a student deviation. The method therefore optimizes imitation on a fixed teacher path, not an on-policy student-generated conversation.

Appendix D. Proposed online, token-at-a-time alter

Design sketch only. It would need a dedicated experiment/config and certification before use.

Objective

Avoid pre-generating and storing the whole teacher trajectory. At token t, obtain a teacher distribution from the RAG-bearing teacher context and a student distribution from the compact context, compute a teacher-sourced KL for that one next-token row, and immediately backpropagate the local layer loss. This changes the memory shape from [answer length, vocabulary] to [1, vocabulary].

Proposed pseudocode — not present in the repository

```
prefix = initial_prompt
for t in range(max_new_tokens):
    p_teacher, teacher_kv = teacher.next_distribution(rag_prompt, prefix, teacher_kv)
    p_student, student_kv = student.next_distribution(compact_prompt, prefix, student_kv)
    local_teacher_kl(p_student, p_teacher).backward()
    token = sample_or_greedy(p_student.detach())
    prefix.append(token)
```

Teacher source and alignment

For a genuinely on-policy variant, append the student-selected token to both histories. The teacher is then queried on the student's actual generated prefix, still with its RAG evidence; this keeps the behavioral target teacher-sourced even after a deviation. Gradients do not pass through the discrete token selection. A teacher cache cannot answer these dynamic-prefix queries, so the teacher must be resident, paged, or recomputed online.

Current online-teacher interface (analogy): src/selfupdate/train/teacher_source.py:75–81

```
t_ids = batch.teacher_ids.to(device)
t_pos = torch.arange(t_ids.shape[1], device=device)[None].expand(
    t_ids.shape[0], -1
)
```

The crucial KV-cache constraint

If `optimizer.step()` occurs after every token, it changes every trainable student block. All student KV entries created under the previous weights are then stale. An exact next-token forward must re-encode the compact prefix after every update; reusing stale KVs would train a different, internally inconsistent model. This is the main compute cost that replaces the current cache-build cost.

Appendix E. KV reuse and teacher-vector directions

Status of three future ideas: distinguish existing code from proposed semantics.

direction	status	meaning
(a) teacher KV minus censored rows	not implemented	Need a defined treatment for later KV rows that already abs
(b) stale student KV after update	not implemented	Avoids replay, but forward uses keys/values made by older v
(c) teacher hidden-vector input	implemented: teacher_ce	Teacher runs full context; student block sees non-privileged

Existing implementation: teacher_censored

The online frozen teacher runs a full sequence and returns raw outputs h0...hn. For each trained block L, the schedule forms the student-visible row index by deleting privileged rows, takes the teacher h[L−1] vectors at those rows as the block input, and targets teacher h[L]. This retains teacher-context information inside the vectors while preventing the student block from directly attending to the deleted token rows.

```
src/selfupdate/train/teacher_source.py:46–58; src/selfupdate/train/layerwise.py:247–253, 289–331

states = [h]
for L in range(1, self.stack.n_layers + 1):
    h = self.stack.run_block(L, h, pos_emb)
    states.append(h)
...
rows = censored_rows(it.s0, tA0, it.A, it.t_priv, device)
# block L consumes censored rows of teacher h[L-1]
```

Why this is not teacher-KV reuse

No training path passes `past_key_values` into the model. The only DynamicCache usage is evaluation-time greedy decoding. A cached teacher key/value from a later row can already contain information mixed from privileged tokens, so simply deleting the privileged KV rows is not equivalent to rerunning a censored teacher attention computation. That ambiguity must be resolved before (a) is a controlled